

JavaScript Tasks — Answers

Template Literals, Default Parameters, Destructuring, Spread & Rest Operators

Task 1

Create a function `formatWelcomeMessage(userName, followers)` that returns a welcome message using a template literal. Call it with your own name and a follower count, then log it.

```
function formatWelcomeMessage(userName, followers) {  
  return `Hello ${userName}, you have ${followers} followers!`;  
}
```

```
console.log(formatWelcomeMessage("Divy", 1200));
```

Output: Hello Divy, you have 1200 followers!

Task 2

Create the same `formatWelcomeMessage(userName, followers)` function using a template literal. Call it and log the result.

```
function formatWelcomeMessage(userName, followers) {  
  return `Hello ${userName}, you have ${followers} followers!`;  
}
```

```
console.log(formatWelcomeMessage("Divy", 1500));
```

Output: Hello Divy, you have 1500 followers!

Task 3

Create a template literal that displays a Spotify-style welcome message. Assign `userName` and `playlistCount` variables and log the message.

```
const userName = "Divy";  
const playlistCount = 8;
```

```
const message = `Hello, ${userName}! You have ${playlistCount} playlists.`;
```

```
console.log(message);
```

Output: Hello, Divy! You have 8 playlists.

Task 4

Write `getDeliveryDetails` with default values of Ahmedabad and UPI. Log both values using a template literal. Call it once with no arguments and once with custom values.

```
function getDeliveryDetails(address = "Ahmedabad", paymentMethod = "UPI") {  
  console.log(`Delivery address: ${address}, Payment method: ${paymentMethod}`);  
}
```

```
getDeliveryDetails();  
getDeliveryDetails("Mumbai", "Cash on Delivery");
```

Output: Delivery address: Ahmedabad, Payment method: UPI Delivery address: Mumbai, Payment method: Cash on Delivery

Task 5

Write the same `getDeliveryDetails` function with default values Ahmedabad and UPI, and test it with no arguments and custom values.

```
function getDeliveryDetails(address = "Ahmedabad", paymentMethod = "UPI") {  
  console.log(`Delivery address: ${address}, Payment method: ${paymentMethod}`);  
}
```

```
getDeliveryDetails();  
getDeliveryDetails("Delhi", "Credit Card");
```

Output: Delivery address: Ahmedabad, Payment method: UPI Delivery address: Delhi, Payment method: Credit Card

Task 6

Write `orderFood(item, quantity = 1)` that returns a string such as 'You ordered 2 Burgers.' Test both the default and custom quantity cases.

```
function orderFood(item, quantity = 1) {  
  return `You ordered ${quantity} ${item}s.`;  
}
```

```
console.log(orderFood("Pizza"));  
console.log(orderFood("Burger", 2));
```

Output: You ordered 1 Pizzas. You ordered 2 Burgers. Note: If you want natural grammar for singular/plural, you can customize the item text. The required default-parameter concept is demonstrated above.

Task 7

Given a Zomato order object, use object destructuring to extract item and price, then log a message using the user's name.

```
const order = {  
  item: "Pizza",  
  price: 350,  
  user: "Amit"  
};
```

```
const { item, price } = order;
```

```
console.log(`${order.user} ordered ${item} for ₹${price}`);
```

Output: Amit ordered Pizza for ₹350

Task 8

Given the same Zomato order object, use object destructuring to extract item and price, then log the required message.

```
const order = {  
  item: "Pizza",  
  price: 350,  
  user: "Amit"  
};
```

```
const { item, price } = order;
```

```
console.log(`${order.user} ordered ${item} for ₹${price}`);
```

Output: Amit ordered Pizza for ₹350

Task 9

Given a movie object, use object destructuring to extract title and rating, then log them in the required format.

```
const movie = {  
  title: "Jawan",  
  rating: 4.5,
```

```
    genre: "Action"
  };

const { title, rating } = movie;

console.log(`Movie: ${title}, Rating: ${rating}`);

Output: Movie: Jawan, Rating: 4.5
```

Task 10

Use the spread operator to combine the cricket-fan and football-fan arrays into allFans and log it.

```
const cricketFans = ["Virat", "Rohit"];
const footballFans = ["Messi", "Ronaldo"];

const allFans = [...cricketFans, ...footballFans];

console.log(allFans);

Output: ["Virat", "Rohit", "Messi", "Ronaldo"]
```

Task 11

Use the spread operator to combine the same two arrays into allFans and log the result.

```
const cricketFans = ["Virat", "Rohit"];
const footballFans = ["Messi", "Ronaldo"];

const allFans = [...cricketFans, ...footballFans];

console.log(allFans);

Output: ["Virat", "Rohit", "Messi", "Ronaldo"]
```

Task 12

Use the spread operator to combine favorite Zomato dishes and favorite Swiggy dishes into allDishes, then log it.

```
const zomatoDishes = ["Pizza", "Biryani", "Burger"];
const swiggyDishes = ["Dosa", "Pasta", "Paneer Tikka"];

const allDishes = [...zomatoDishes, ...swiggyDishes];

console.log(allDishes);

Output: ["Pizza", "Biryani", "Burger", "Dosa", "Pasta", "Paneer Tikka"]
```

Task 13

Refactor the given function into an arrow function using rest parameters. The new function must be called addToCartArrow and return the sum of any number of prices.

```
const addToCartArrow = (...prices) => {
  let total = 0;

  for (let price of prices) {
    total += price;
  }

  return total;
};

console.log(addToCartArrow(100, 250, 50));

Output: 400
```

Task 14

Refactor the same function into an arrow function using rest parameters. Call the new function `addToCartArrow` and return the sum.

```
const addToCartArrow = (...prices) => {  
  let total = 0;  
  
  for (let price of prices) {  
    total += price;  
  }  
  
  return total;  
};  
  
console.log(addToCartArrow(120, 80, 300, 50));  
Output: 550
```

Task 15

Write an arrow function called `getFollowersSummary` that accepts any number of follower counts using the rest operator, sums them, and returns a string such as 'Total followers: 1200'. Test it with three sample numbers.

```
const getFollowersSummary = (...followerCounts) => {  
  const total = followerCounts.reduce(  
    (sum, count) => sum + count,  
    0  
  );  
  
  return `Total followers: ${total}`;  
};  
  
console.log(getFollowersSummary(400, 350, 450));  
Output: Total followers: 1200
```